

Managing Security Settings

Kubernetes Production · Lesson 14

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Trace a request through **authentication** → **authorization** → **admission**
- Give Pods an identity and a token with **ServiceAccounts**
- Build **RBAC** rules with Roles, ClusterRoles and their bindings
- Apply the **principle of least privilege** to subjects
- Verify access with `kubectl auth can-i`
- Harden Pods and containers with **securityContext**

Reference: *Using RBAC Authorization* — kubernetes.io/docs/reference/access-authn-authz/rbac

14.1 The Request Flow

Authentication → Authorization → Admission

Every request to the **kube-apiserver** passes through three stages:

```
client (kubectl, Pod, controller)
  |
  ▼
[1] Authentication —▶ Who are you? (User / Group / ServiceAccount)
  |
  ▼
[2] Authorization —▶ Are you allowed? (RBAC checks verb + resource)
  |
  ▼
[3] Admission      —▶ Mutate / validate the object, then persist to etcd
```

- Fail any stage → the request is **rejected** before it is persisted.
- **RBAC** is the authorization mode you configure and test on the CKA.

Identities: Users vs ServiceAccounts

Kubernetes distinguishes two kinds of identity:

Identity	For	Managed by
User / Group	Humans, external clients	Outside the cluster (certs, OIDC) — no API object
ServiceAccount	Processes inside Pods	A real namespaced API object

- There is **no User object** in Kubernetes — users come from certificates, tokens or an identity provider.
- **RBAC is additive only** — there are no deny rules; no grant means no access.

14.2 ServiceAccounts

The default ServiceAccount

- Every namespace contains at least one ServiceAccount named `default`.
- A Pod with `no serviceAccountName` is assigned `default` automatically.
- By default the kubelet **automounts** a token at `/var/run/secrets/kubernetes.io/serviceaccount/token`.

```
kubectl get serviceaccounts
kubectl get pod my-pod -o jsonpath='{.spec.serviceAccountName}'
```

CKA tip: the `default` SA has **almost no permissions** — grant access by binding a Role to it, never by editing the SA itself.

Creating a ServiceAccount & tokens

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: build-robot
  namespace: default
automountServiceAccountToken: false # opt out of token mounting
```

```
kubectl create serviceaccount build-robot
kubectl create token build-robot --duration=10m # bound, time-limited token
```

- `kubectl create token` issues a **bound, time-limited** token (v1.22+).
- Set `automountServiceAccountToken: false` when a Pod needs **no** API access.

Projected tokens & automounting

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  serviceAccountName: build-robot      # use a specific SA
  automountServiceAccountToken: false  # Pod-level override wins
  containers:
  - name: app
    image: nginx
```

- Modern tokens are **projected**: short-lived, audience-bound, auto-rotated, and **invalidated** when the Pod is deleted.
- `automountServiceAccountToken` set on the **SA** or the **Pod** — the **Pod spec takes precedence**.
- `serviceAccountName` is **immutable** after the Pod is created.

14.3 RBAC Objects

The four RBAC objects

RBAC uses the API group `rbac.authorization.k8s.io/v1` with four objects:

Object	Scope	Defines / Grants
Role	Namespace	Permissions within one namespace
ClusterRole	Cluster	Permissions cluster-wide / on cluster-scoped resources
RoleBinding	Namespace	Grants a Role or ClusterRole in one namespace
ClusterRoleBinding	Cluster	Grants a ClusterRole across all namespaces

Roles/ClusterRoles **define** permissions; Bindings **grant** them to subjects.

Role vs ClusterRole

Aspect	Role	ClusterRole
Scope	Single namespace	Cluster-wide
Namespaced resources	Yes (that namespace)	Yes (any / all namespaces)
Cluster-scoped resources (nodes, PVs)	No	Yes
Non-resource URLs (/healthz)	No	Yes
Reusable across namespaces	No	Yes (via RoleBinding)

- Use a **Role** for "edit Pods in team-a".
- Use a **ClusterRole** for "read nodes" or a permission reused everywhere.

Rules: apiGroups / resources / verbs

A rule answers **what** can be done to **which** resources:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]                # "" = the core API group
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

- Common **verbs**: `get`, `list`, `watch`, `create`, `update`, `patch`, `delete`, `deletecollection`.
- `resourceNames` (optional) restricts a rule to **named** objects only.

A ClusterRole & built-in roles

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: secret-reader      # no namespace – cluster-scoped
rules:
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get", "watch", "list"]
```

- Identical rule syntax to a Role — only the **scope** differs.

Kubernetes ships default ClusterRoles: **cluster-admin**, **admin**, **edit**, **view** — reuse these before writing your own.

RoleBinding: subjects + roleRef

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User                # User, Group, or ServiceAccount
  name: jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

- **subjects** = who; **roleRef** = which Role/ClusterRole.
- **roleRef** is **immutable** — delete and recreate to change it.

Binding a SA & reusing a ClusterRole

A RoleBinding can reference a **ClusterRole**, scoping it to one namespace:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-secrets
  namespace: development
subjects:
- kind: ServiceAccount
  name: build-robot
  namespace: development
roleRef:
  kind: ClusterRole           # reused, but only in 'development'
  name: secret-reader
  apiGroup: rbac.authorization.k8s.io
```

- A **ClusterRoleBinding** instead would grant `secret-reader` in **all** namespaces.

Creating RBAC with kubectl

```
# Imperative Role + RoleBinding (great for the exam)
kubectl create role pod-reader \
  --verb=get,list,watch --resource=pods

kubectl create rolebinding read-pods \
  --role=pod-reader --user=jane --namespace=default

# Bind a ClusterRole to a ServiceAccount, cluster-wide
kubectl create clusterrolebinding robot-view \
  --clusterrole=view \
  --serviceaccount=default:build-robot
```

CKA tip: `--serviceaccount` takes the form `<namespace>:<name>` .

14.4 Least Privilege & Verifying

Principle of least privilege

Grant **only** the permissions a subject actually needs:

- Prefer a **Role** in one namespace over a cluster-wide grant.
- List **specific resources and verbs** — avoid `resources: ["*"]`, `verbs: ["*"]`.
- Bind to a **dedicated ServiceAccount**, not the `default` SA.
- Use `resourceNames` to scope down to named objects when possible.
- Reuse the built-in **view / edit** ClusterRoles before inventing broad ones.

RBAC blocks **privilege escalation**: you can only grant permissions you already hold (or that you're explicitly allowed to grant via `escalate / bind`). Reserve **cluster-admin** for break-glass use.

Verifying access: `kubectl auth can-i`

```
# Check your own permissions
kubectl auth can-i create deployments --namespace=dev
kubectl auth can-i delete pods

# Impersonate a user or ServiceAccount (admin only)
kubectl auth can-i list secrets \
  --as=system:serviceaccount:default:build-robot

kubectl auth can-i get nodes --as=jane --as-group=developers
```

- Answers a plain `yes` or `no`.
- `--as` / `--as-group` impersonate a subject to test their effective rights.

14.5 Security Contexts

securityContext: pod vs container

A `securityContext` constrains how container processes run on the node:

- **Pod-level** (`spec.securityContext`) — defaults for **all** containers.
- **Container-level** (`spec.containers[].securityContext`) — **overrides** the Pod-level value for that container.

Field	Level	Effect
<code>runAsUser</code> / <code>runAsGroup</code>	Pod + Container	UID / primary GID of processes
<code>runAsNonRoot</code>	Pod + Container	Refuse to start if image runs as root
<code>fsGroup</code>	Pod only	Group owner for mounted volumes
<code>capabilities</code>	Container only	Add / drop Linux capabilities

Pod-level securityContext

```
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
spec:
  securityContext:
    runAsUser: 1000      # all processes run as UID 1000
    runAsGroup: 3000     # primary group GID 3000
    fsGroup: 2000        # volumes owned by GID 2000
  containers:
    - name: app
      image: busybox:1.28
      command: ["sh", "-c", "sleep 1h"]
```

- `fsGroup` makes mounted volume files group-owned by `2000` (sets setgid).
- These settings apply to **every** container unless overridden.

Hardening a container

```
securityContext:
  runAsNonRoot: true           # block running as root
  runAsUser: 2000
  allowPrivilegeEscalation: false # sets no_new_privs
  readOnlyRootFilesystem: true   # immutable root FS
  capabilities:
    drop: ["ALL"]              # remove every capability
    add: ["NET_BIND_SERVICE"]  # then add only what is needed
  seccompProfile:
    type: RuntimeDefault
```

- Drop **ALL** capabilities, then add back the minimum.
- `allowPrivilegeEscalation: false` and `readOnlyRootFilesystem: true` are strong, low-cost hardening defaults.

CKA tip: `privileged: true` and `CAP_SYS_ADMIN` **force** `allowPrivilegeEscalation` back to true — avoid them.

Key Takeaways

- Requests pass **authentication** → **authorization** → **admission**.
- **ServiceAccounts** identify Pods; `default` exists per namespace, tokens are projected and automounting is opt-out (Pod spec wins).
- **RBAC** = Role/ClusterRole (define) + RoleBinding/ClusterRoleBinding (grant), on `rbac.authorization.k8s.io/v1`.
- A **RoleBinding** can reference a **ClusterRole** to reuse it per namespace.
- Apply **least privilege**; verify with `kubectl auth can-i`.
- Harden with **securityContext**: `runAsNonRoot`, `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem`, drop **ALL** capabilities.

End of Lesson 14

Next: Lesson 15 — CRDs & Operators

```
kubectl auth can-i · kubectl create role · kubectl create rolebinding
```